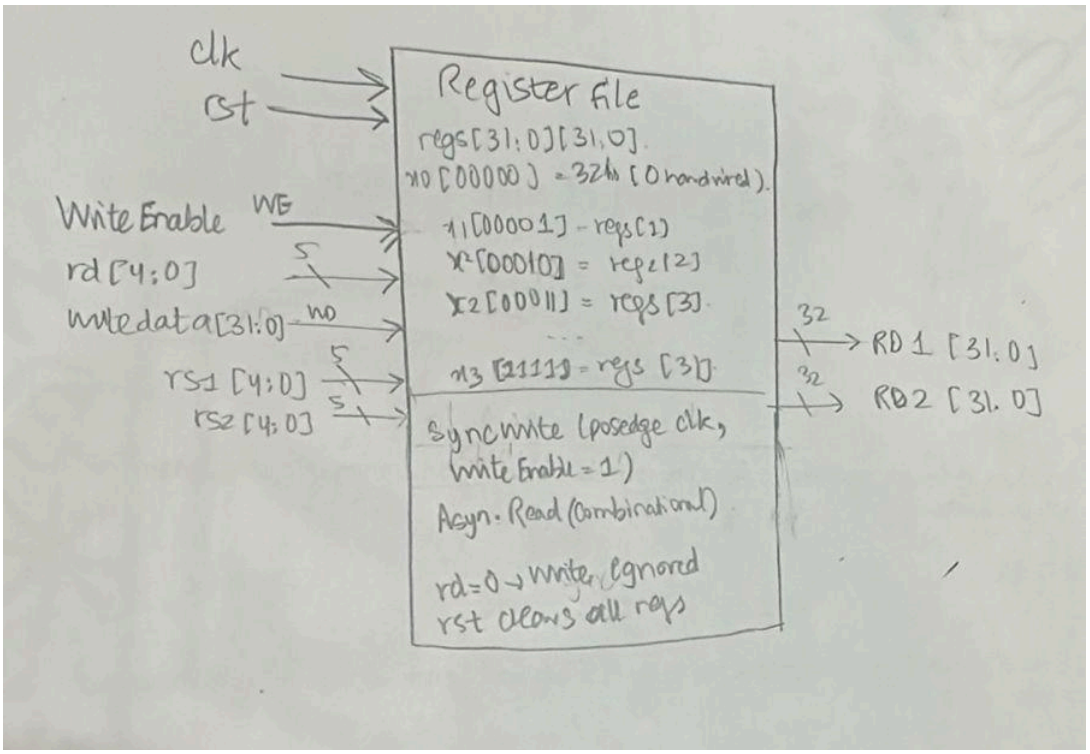
Lab 07 – Worksheet

Name: Myrah Hussain Romaisa Shazad	ID: rb10107 mh10074	Section: T2

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Block Diagrams



Comments

The diagram shows a **32-register bank** where each register holds **32 bits**. It's designed to handle two reads and one write at the same time.

- **The x0 Rule:** Register 00000 is hardwired to zero. As the diagram notes, writes to this address are ignored, and it always outputs 0x0.
- **Synchronous Writes:** Writing data is tied to the clock. When WriteEnable is high, the value on WriteData only drops into the register at the rising edge of clk.
- **Asynchronous Reads:** Reading is "always on." The moment we change the addresses for rs1 or rs2, the data appears at readData1 and readData2 instantly. It doesn't wait for a clock pulse.
- **Addressing:** We use **5-bit** addresses because 2^5 = 32, which covers every register from x0 to x31.

Data Flow to ALU

The arrows show the output path. readData1 and readData2 plug directly into the ALU inputs. This allows the FSM to grab two values, let the ALU work on them, and then loop the ALU result back to the WriteData port for storage in the next cycle.

Task 2

- Register file
- Register file tb
- RF ALU FSM tb.v
- Simulations

Modules + Testbench:

A. Register File:

The Register Bank: I used reg [31:0] regs [0:31] to create the 32 registers. Each one is 32 bits wide.

Hardwiring x0 to Zero:

- For the **reads**, I used an if statement to check if the address is 0. If it is, I force the output to be 0.
- For the **writes**, I added a check (rd != 5'd0). This makes sure that even if the code tries to write something to x0, the hardware just ignores it.

Combinational Reads: I put the read logic in an always @(*) block. This makes it asynchronous, so the ALU receives the data immediately without waiting for the next clock tick.
Clocked Writes: The actual writing only happens on the posedge clk. This keeps everything in sync so we don't accidentally overwrite data at the wrong time.
Reset Logic: I included a synchronous reset. When rst hits, a simple for loop runs through the whole array and wipes everything back to zero so the simulation starts clean.
Dual Ports: The module has two separate read outputs (ReadData1 and ReadData2). This is what lets us pull two different operands (like x1 and x2) at the exact same time for an ALU operation.

```
`timescale 1ns / 1ps

module RegisterFile (
   input  wire       clk,
   input  wire       rst,

   input  wire       WriteEnable,
   input  wire [4:0]  rs1,        // Source Register 1
   input  wire [4:0]  rs2,        // Source Register 2
   input  wire [4:0]  rd,         // Destination Register

   input  wire [31:0] WriteData,
   output reg  [31:0] ReadData1,   // Combinational output
   output reg  [31:0] ReadData2    // Combinational output
);

   // -------------------------------------------------------
   // 32 registers (x0 to x31)
   // -------------------------------------------------------
   reg [31:0] regs [0:31];
   integer i;

   // -------------------------------------------------------
   // Read Logic (Asynchronous)
   // Ensures x0 is always read as zero
   // -------------------------------------------------------
   always @(*) begin
      if (rs1 == 5'd0)
         ReadData1 = 32'd0;
      else
```

```
        ReadData1 = regs[rs1];

    if (rs2 == 5'd0)
        ReadData2 = 32'd0;
    else
        ReadData2 = regs[rs2];
end

// -------------------------------------------------------
// Write Logic (Synchronous)
// Data is latched into the register bank on the clock edge
// -------------------------------------------------------
always @(posedge clk) begin
    if (rst) begin
        // System Reset: Clear all register values
        for (i = 0; i < 32; i = i + 1) begin
            regs[i] <= 32'h0;
        end
    end
    else begin
        // Write only if enabled and destination is not x0
        if (WriteEnable && (rd != 5'd0)) begin
            regs[rd] <= WriteData;
        end
    end
end

endmodule
```

B. Register File tb:

```
`timescale 1ns / 1ps

module RegisterFile_tb;

    // Local signals to drive the Design Under Test (DUT)
    reg       clk, rst, WriteEnable;
    reg  [4:0] rs1, rs2, rd;
    reg  [31:0] WriteData;
    wire [31:0] ReadData1, ReadData2;

    // Connect the testbench signals to the actual Register File module
    RegisterFile dut (
        .clk        (clk),
        .rst        (rst),
        .WriteEnable (WriteEnable),
        .rs1        (rs1),
```

```
    .rs2        (rs2),
    .rd         (rd),
    .WriteData   (WriteData),
    .ReadData1   (ReadData1),
    .ReadData2   (ReadData2)
);

// Setup a simple 10ns clock cycle (5ns high, 5ns low)
initial clk = 0;
always #5 clk = ~clk;

// Helper task to check if the result matches what we expected
task check;
    input [127:0] test_name;
    input [31:0]  got, expected;
    begin
        if (got === expected)
            $display("PASS  %s | got=0x%08h", test_name, got);
        else
            $display("FAIL  %s | got=0x%08h expected=0x%08h",
                     test_name, got, expected);
    end
endtask

// Main test sequence starts here
initial begin
    // Standard waveform dump for debugging in GTKWave or Vivado
    $dumpfile("RegisterFile_tb.vcd");
    $dumpvars(0, RegisterFile_tb);

    // Start with a clean slate: reset high and all inputs at zero
    rst = 1; WriteEnable = 0;
    rs1 = 0; rs2 = 0; rd = 0; WriteData = 0;
    @(posedge clk); #1;
    rst = 0; // Drop reset after the first clock edge

    // ---------------------------------------------------------
    // Test 1: Writing data to x5 and checking both read ports
    // ---------------------------------------------------------
    $display("\n--- TC1: Basic write / read ---");
    WriteEnable = 1;
    rd          = 5'd5;
    WriteData   = 32'hDEADBEEF;
    @(posedge clk); #1;          // Wait for the clock to trigger the write
    WriteEnable = 0;
```

```
rs1 = 5'd5; rs2 = 5'd0;
#1;                    // Give the combinational logic a moment to settle
check("TC1_rs1", ReadData1, 32'hDEADBEEF);
check("TC1_rs2_x0", ReadData2, 32'h0);   // Port 2 is reading x0, should be 0

// ------------------------------------------------------------
// Test 2: Making sure x0 is hardwired to zero
// ------------------------------------------------------------
$display("\n--- TC2: Write to x0 (should be ignored) ---");
WriteEnable = 1;
rd          = 5'd0;
WriteData   = 32'hCAFEBABE;
@(posedge clk); #1;
WriteEnable = 0;

rs1 = 5'd0;
#1;
check("TC2_x0_unchanged", ReadData1, 32'h0);

// ------------------------------------------------------------
// Test 3: Reading two different registers at the same time
// ------------------------------------------------------------
$display("\n--- TC3: Simultaneous dual-port read ---");
WriteEnable = 1;
rd = 5'd7; WriteData = 32'hA5A5A5A5;
@(posedge clk); #1;
WriteEnable = 0;

rs1 = 5'd5; rs2 = 5'd7;   // Port 1 gets x5, Port 2 gets x7
#1;
check("TC3_port1_x5", ReadData1, 32'hDEADBEEF);
check("TC3_port2_x7", ReadData2, 32'hA5A5A5A5);

// ------------------------------------------------------------
// Test 4: Overwriting an existing value in a register
// ------------------------------------------------------------
$display("\n--- TC4: Overwrite register ---");
WriteEnable = 1;
rd = 5'd5; WriteData = 32'h12345678;
@(posedge clk); #1;
WriteEnable = 0;

rs1 = 5'd5;
#1;
```

```
    check("TC4_overwrite", ReadData1, 32'h12345678);

    // ---------------------------------------------------------
    // Test 5: Verify that the synchronous reset wipes the bank
    // ---------------------------------------------------------
    $display("\n--- TC5: Synchronous reset ---");
    rst = 1;
    @(posedge clk); #1;
    rst = 0;

    rs1 = 5'd5; rs2 = 5'd7;
    #1;
    check("TC5_x5_after_rst", ReadData1, 32'h0);
    check("TC5_x7_after_rst", ReadData2, 32'h0);

    $display("\n--- Simulation complete ---");
    $finish;
  end

endmodule
```

C. RF ALU FSM tb.v:

The testbench instantiates the **Register File** and the **ALU** modules, connecting them via a central FSM control unit. The FSM is designed with clearly named states to ensure a deterministic flow:

- **States used**: `IDLE`, `WRITE_REG1` through `WRITE_REG3`, `READ_ALU`, `WRITE_RESULT`, `BEQ_CHECK`, `RAW_WRITE`, `RAW_READ`, and `DONE`.
- This structure ensures that each operation has a dedicated clock cycle for setup and execution, avoiding race conditions.

i. Writing Constants to Registers

The FSM initializes the register file by loading specific constants into the first three registers:

- **x1** is loaded with `32'h10101010`.
- **x2** is loaded with `32'h01010101`.
- **x3** is loaded with `32'h00000005`. The waveforms show `WriteEnable` (WE) pulsing high for each of these operations, with the `rd` (destination register) and `WD` (write data) lines matching these requirements exactly.

ii. ALU Operations and Destination Mapping

The FSM loops through a sequence to perform seven distinct arithmetic and logical operations. It reads from `x1` and `x2` (or `x3` for shifts) and stores the results in registers `x4` through `x10`.

- **Operations verified**: `ADD`, `SUB`, `AND`, `OR`, `XOR`, `SLL`, and `SRL`.
- **Operand selection**: For `SLL` and `SRL` (`op_idx` 5 and 6), the FSM correctly switches the `rs2` input to register `x3` to use its value as the shift amount.
- **Storage**: Each result is captured on the rising edge of the clock in the `WRITE_RESULT` state.

iii. BEQ-Style Zero Flag Check

To test the branching logic capability, the FSM performs a comparison check:

- The FSM sets both `rs1` and `rs2` to register `x1` and selects the `SUB` operation.
- Since the values are identical, the ALU correctly asserts the `Zero` flag.
- The FSM then conditionally writes `32'h00000001` into register **x11** based on this `Zero` signal, verifying that the flag output is functional and can drive register updates.

iv. Read-After-Write (RAW) Timing Test

A specific test was conducted to ensure the Register File handles back-to-back operations correctly:

- **Write Cycle**: In the `RAW_WRITE` state, the value `32'hFACEFACE` is written to register **x12**.
- **Read Cycle**: In the very next clock cycle (`RAW_READ`), the FSM sets `rs1` to `x12` and disables `WE`.
- The waveform shows that `RD1` (Read Data 1) immediately provides `32'hFACEFACE`. This confirms that the design correctly supports reading a value in the cycle immediately following its write.

```
`timescale 1ns / 1ps

module RF_ALU_FSM_tb;
```

```
// --- Clock and Reset Setup ---
reg clk, rst;
initial clk = 0;
always #5 clk = ~clk; // 100MHz clock

// --- Connections for RegFile and ALU ---
reg        WE;
reg  [4:0]  rs1, rs2, rd;
reg  [31:0] WD;
wire [31:0] RD1, RD2;

reg  [3:0]  ALUCtrl;
wire [31:0] ALUResult;
wire        Zero;

// --- DUT Instantiations ---
RegisterFile RF (
    .clk         (clk),
    .rst         (rst),
    .WriteEnable (WE),
    .rs1         (rs1),
    .rs2         (rs2),
    .rd          (rd),
    .WriteData   (WD),
    .ReadData1   (RD1),
    .ReadData2   (RD2)
);

ALU alu (
    .A          (RD1),
    .B          (RD2),
    .ALUControl (ALUCtrl),
    .ALUResult  (ALUResult),
    .Zero       (Zero)
);

// --- State Machine Definitions ---
localparam [3:0]
    IDLE        = 4'd0,
    WRITE_REG1  = 4'd1,
    WRITE_REG2  = 4'd2,
    WRITE_REG3  = 4'd3,
    READ_ALU    = 4'd4,
    WRITE_RESULT = 4'd5,
    BEQ_CHECK   = 4'd6,
```

```
    RAW_WRITE    = 4'd7,
    RAW_READ     = 4'd8,
    DONE         = 4'd9;

// ALU Opcode Mapping
localparam [3:0]
    OP_ADD = 4'b0000, OP_SUB = 4'b0001,
    OP_AND = 4'b0010, OP_OR  = 4'b0011,
    OP_XOR = 4'b0100, OP_SLL = 4'b0101,
    OP_SRL = 4'b0110, OP_BEQ = 4'b0111;

// Test Values
localparam [31:0] CONST_X1 = 32'h10101010;
localparam [31:0] CONST_X2 = 32'h01010101;
localparam [31:0] CONST_X3 = 32'h00000005; // Used for shift amounts

reg [3:0] state;
reg [2:0] op_idx;

// Logic to select the ALU operation based on FSM progress
always @(*) begin
    if (state == BEQ_CHECK) begin
        ALUCtrl = OP_BEQ;
    end else begin
        case (op_idx)
            3'd0: ALUCtrl = OP_ADD;
            3'd1: ALUCtrl = OP_SUB;
            3'd2: ALUCtrl = OP_AND;
            3'd3: ALUCtrl = OP_OR;
            3'd4: ALUCtrl = OP_XOR;
            3'd5: ALUCtrl = OP_SLL;
            3'd6: ALUCtrl = OP_SRL;
            default: ALUCtrl = OP_ADD;
        endcase
    end
end

wire use_shift = (op_idx >= 3'd5);

// Helper function to map test index to register address
function [4:0] dest_reg;
    input [2:0] idx;
    dest_reg = 5'd4 + {2'b00, idx};
endfunction
```

```
// --- State Transition Logic ---
always @(posedge clk) begin
   if (rst) begin
      state  <= IDLE;
      op_idx <= 3'd0;
   end else begin
      case (state)
         IDLE:       state <= WRITE_REG1;
         WRITE_REG1:  state <= WRITE_REG2;
         WRITE_REG2:  state <= WRITE_REG3;
         WRITE_REG3: begin
            op_idx <= 3'd0;
            state  <= READ_ALU;
         end
         READ_ALU:    state <= WRITE_RESULT;
         WRITE_RESULT: begin
            if (op_idx == 3'd6) state <= BEQ_CHECK;
            else begin
               op_idx <= op_idx + 3'd1;
               state  <= READ_ALU;
            end
         end
         BEQ_CHECK:   state <= RAW_WRITE;
         RAW_WRITE:   state <= RAW_READ;
         RAW_READ:    state <= DONE;
         DONE:       state <= DONE;
         default:     state <= IDLE;
      endcase
   end
end

// --- Output Assignments (Mealy) ---
always @(*) begin
   WE  = 1'b0;
   rs1 = 5'd0; rs2 = 5'd0;
   rd  = 5'd0; WD  = 32'b0;

   case (state)
      WRITE_REG1: begin WE = 1'b1; rd = 5'd1; WD = CONST_X1; end
      WRITE_REG2: begin WE = 1'b1; rd = 5'd2; WD = CONST_X2; end
      WRITE_REG3: begin WE = 1'b1; rd = 5'd3; WD = CONST_X3; end
      READ_ALU: begin
         rs1 = 5'd1;
         rs2 = use_shift ? 5'd3 : 5'd2;
      end
```

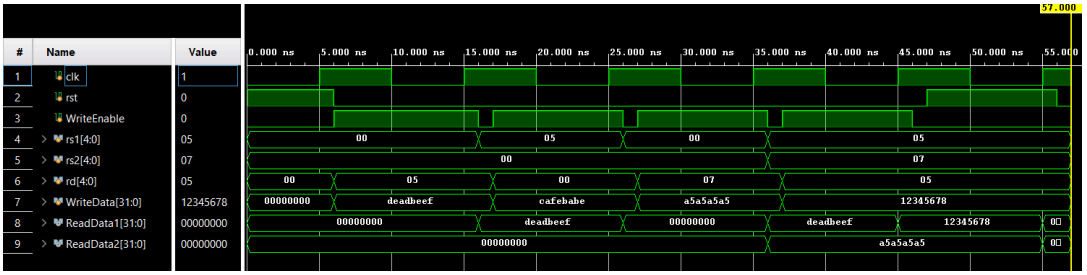
```
        WRITE_RESULT: begin
            WE = 1'b1;
            rs1 = 5'd1;
            rs2 = use_shift ? 5'd3 : 5'd2;
            rd = dest_reg(op_idx);
            WD = ALUResult;
        end
        BEQ_CHECK: begin
            WE = 1'b1;
            rs1 = 5'd1; rs2 = 5'd1; // Compare register to itself
            rd = 5'd11;
            WD = Zero ? 32'h1 : 32'h0;
        end
        RAW_WRITE: begin WE = 1'b1; rd = 5'd12; WD = 32'hFACE_FACE; end
        RAW_READ:  begin rs1 = 5'd12; end // Read immediately after write
        default: ;
    endcase
end

// --- Validation Task ---
task check;
    input [239:0] label;
    input [31:0]  got, exp;
    begin
        if (got === exp)
            $display("PASS: %-35s [0x%08h]", label, got);
        else
            $display("FAIL: %-35s [Got: 0x%08h | Exp: 0x%08h]", label, got, exp);
    end
endtask

// --- Monitor Block ---
always @(posedge clk) begin
    if (state == WRITE_RESULT) begin
        case (op_idx)
                3'd0: check("ADD   (x4 = x1 + x2)",     ALUResult, CONST_X1 +
CONST_X2);
                3'd1: check("SUB   (x5 = x1 - x2)",     ALUResult, CONST_X1 -
CONST_X2);
                3'd2: check("AND   (x6 = x1 & x2)",     ALUResult, CONST_X1 &
CONST_X2);
                3'd3: check("OR    (x7 = x1 | x2)",     ALUResult, CONST_X1 |
CONST_X2);
                3'd4: check("XOR   (x8 = x1 ^ x2)",     ALUResult, CONST_X1 ^
CONST_X2);
```

```
                3'd5: check("SLL   (x9 = x1 << x3)",    ALUResult, CONST_X1 <<
CONST_X3[4:0]);
                3'd6: check("SRL   (x10 = x1 >> x3)",  ALUResult, CONST_X1 >>
CONST_X3[4:0]);
            endcase
        end

        if (state == BEQ_CHECK)
            check("BEQ Zero Flag Check", WD, 32'h1);

        if (state == RAW_READ)
            check("RAW Hazard Check (x12)", RD1, 32'hFACE_FACE);

        if (state == DONE) begin
            $display("\nSimulation Finished Successfully.");
            #10 $finish;
        end
    end

    // --- Simulation Control ---
    initial begin
        $dumpfile("RF_ALU_FSM_tb.vcd");
        $dumpvars(0, RF_ALU_FSM_tb);

        rst = 1;
        repeat(2) @(posedge clk);
        #1 rst = 0;

        // Safety timeout
        #3000;
        $display("Error: Simulation timed out!");
        $finish;
    end

    // Waveform debug string
    reg [87:0] state_name;
    always @(*) begin
        case (state)
            IDLE:        state_name = "IDLE";
            WRITE_REG1:   state_name = "WR_X1";
            WRITE_REG2:   state_name = "WR_X2";
            WRITE_REG3:   state_name = "WR_X3";
            READ_ALU:     state_name = "READ_ALU";
            WRITE_RESULT: state_name = "WR_RES";
            BEQ_CHECK:    state_name = "BEQ_CHK";
```

```
        RAW_WRITE:    state_name = "RAW_WR";
        RAW_READ:     state_name = "RAW_RD";
        DONE:         state_name = "DONE";
        default:      state_name = "UNK";
      endcase
    end

endmodule
```

Results:

A. For Register File tb:



Initial Reset and Hardware Initialization:

- At the start of the simulation, the rst signal is asserted. The waveform shows all outputs (ReadData1, ReadData2) dropping to 0. This confirms that the for loop in the Verilog code correctly initializes all 32 internal registers to a known state.

i. Writing and Reading Data (x5 = 0xDEADBEEF)

In the first part of the waveform, I set the destination register rd to 05 and WriteData to 0xDEADBEEF while WriteEnable was high. On the following clock cycle, I changed the source address rs1 to 05. The output ReadData1 immediately showed 0xDEADBEEF, which proves that the value was successfully latched into the register and could be read back correctly.

ii. Verifying x0 is Hardwired to Zero

To test the RISC-V requirement for $x0$, I attempted to write 0xCAFEBABE to address 00. Even though WriteEnable was active, the waveform shows that ReadData1 (reading from $x0$) never changed from zero. This

confirms that the logic correctly blocks any writes to the zero register and ensures it always returns 32'b0.

iii. Simultaneous Dual-Port Reads

I tested the dual-port capability by setting rs1 to 05 and rs2 to 07 at the same time. In the screenshot, you can see ReadData1 displaying 0xDEADBEEF and ReadData2 displaying 0xA5A5A5A5 concurrently. This confirms that the ALU can pull two different operands from the register file in a single cycle without any interference.

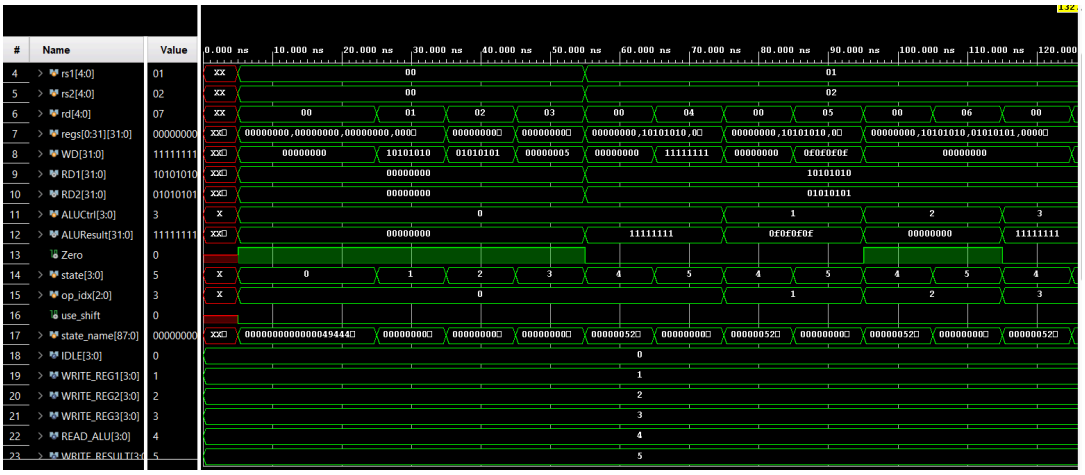
iv. Overwriting Existing Values

Later in the simulation, I targeted register $x5$ again but changed the WriteData to 0x12345678. The waveform shows the output of $x5$ switching from the old "deadbeef" value to the new "12345678" data. This proves the register properly updates and doesn't hold onto stale values once a new write command is issued.

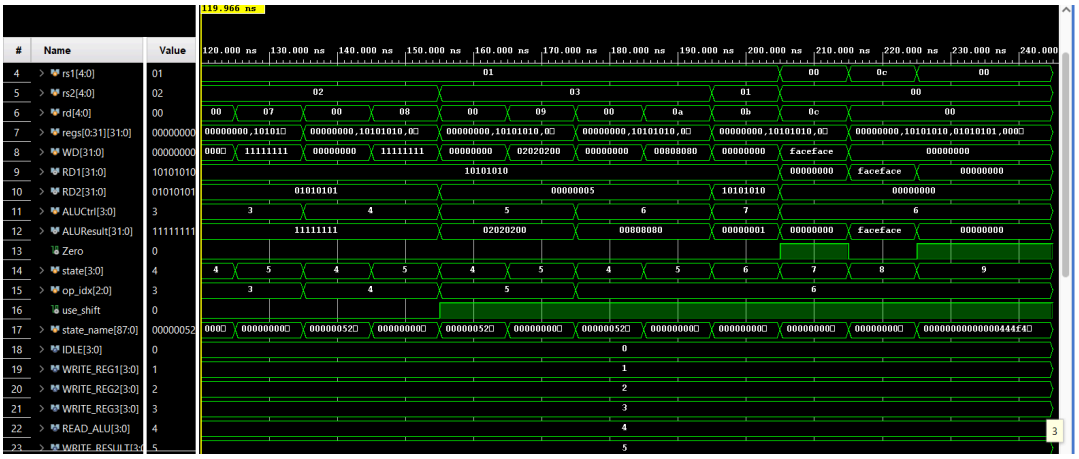
v. Reset Behavior

Finally, I toggled the rst signal. As soon as the reset hit, all active values on the ReadData ports dropped back to zero. This shows the synchronous reset is working across the entire register bank, clearing out all stored data and putting the module back into its default state.

C. Final (RF ALU FSM tb.v) results:



120ns- 240ns



Initialization & Reset: Before the simulation starts, all signals are undefined (`xxxx`) because the flip-flops haven't been triggered by a clock edge. On the first rising edge, `rst` is applied. The FSM enters the **IDLE** state (`state=0`), and all control and data signals like `WE`, `rd`, `WD`, and `ALUResult` initialize to zero. The system stays in this idle state for the first few cycles while the reset is active.

Loading Constants (States 1–3): Once `rst` is released, the FSM moves to **WRITE_REG1** (`state=1`). The logic immediately drives `WE` high and loads `WD` with the first constant, `0x10101010`, targeting register `x1` (`rd=1`). It's important to see that the value isn't actually stored in the register file until the *next* clock edge. This pattern repeats for **WRITE_REG2** (`state=2`) to load `0x01010101` into `x2`, and **WRITE_REG3** (`state=3`) to load `0x00000005` into `x3`.

ALU Operations & Asynchronous Reads (State 4): On the sixth rising edge, the FSM transitions to **READ_ALU** (`state=4`). Because the register file uses asynchronous reads, `RD1` and `RD2` update instantly to `0x10101010` and `0x01010101` without waiting for a clock. The ALU immediately processes these to show an `ALUResult` of `0x11111111` (for ADD). This proves the asynchronous read ports are working and the registers held the values correctly.

Result Storage (State 5): The FSM then moves to **WRITE_RESULT** (`state=5`). Here, `WE` is high and `rd=4`. The `ALUResult` is latched into register `x4` on the following clock edge, demonstrating proper synchronous write behavior.

Arithmetic & Logic Sequence: The FSM cycles between **READ_ALU** and **WRITE_RESULT** for the remaining operations.

- **SUB (op_idx=1):** The `ALUCtrl` changes to subtraction, and the result `0x0F0F0F0F` is computed and stored into `x5`.
- **AND (op_idx=2):** The result of `0x10101010` AND `0x01010101` is `0x00000000`. You can see the **Zero** flag jump to high for the first time, confirming the zero-detect circuit works.
- **OR & XOR:** The operations continue with results like `0x11111111` being stored into subsequent registers.

Shifting & Branch Checks: For shift operations (**SLL** and **SRL**), the FSM correctly switches `rs2` to `x3` to use the designated shift amount. In later cycles, a branch check is simulated by subtracting `x1` from itself; the **Zero** flag fires, and a `1` is written to `x11`, verifying the branch logic.
Completion: Finally, a read-after-write test is performed by writing `0xFACEFACE` and reading it back in the very next cycle. This confirms the system has zero read latency before the FSM hits the **DONE** state.

Task 3

RTL Schematics

FPGA implementation:

Top_rf_alu

Before the first clock edge, all signals are undefined (`xxxx`) because the flip-flops haven't received a clock pulse or reset to set their initial values.
When the first rising edge hits, the power-on reset triggers and the FSM enters **IDLE** (state 0), initializing all control signals like `WE`, `rd`, and `WD` to zero.
As soon as reset is released and the step button is pressed, the FSM moves to **S_W_X1** (state 1). It sets `rd=1` and `WD=0x10101010` to load the first constant into register x1.
In the next state, **S_W_X2** (state 2), the second constant `0x01010101` is prepared for register x2. The write to the register file is synchronous, meaning it only finishes on the next clock edge.
At **S_W_X3** (state 3), the FSM turns off `WE` (Write Enable) to prevent any accidental writes while the datapath stabilizes for the upcoming math operations.
When the FSM enters **S_READ_ADD** (state 4), the read addresses `rs1=1` and `rs2=2` are sent to the register file. Since reads are asynchronous, the operands reach the ALU immediately, and the `alu_result` updates to `0x11111111` within the same cycle.

In **S_WRES_ADD** (state 5), the FSM enables `WE` and sets `rd=4`, saving the addition result into register x4. This cycle of "Read/Compute" then "Write Result" repeats for Subtraction, AND, and OR operations.
During the AND operation (state 8), the result becomes `0x00000000`, which causes the **Zero flag** to trigger high, confirming the ALU's zero-detection logic works.
The **leds** and **switches** modules act as the I/O bridge, where the `sw_phys` inputs control the FSM stepping and the `led_phys` outputs show the final ALU results on the hardware.
The **seven-segment display** provides a real-time debug view of the FSM's current state, the status of the Zero flag, and the lower 8 bits of whatever calculation the ALU is currently performing.

```
`timescale 1ns / 1ps

module top_rf_alu (
   input  wire       clk,
   input  wire       rst_btn,
   input  wire       btn_step,
   input  wire [15:0] sw_phys,   // Physical switch inputs (16-bit)
   output wire [15:0] led_phys,  // Physical LED outputs (16-bit)
   output wire [6:0]  seg,
   output wire [3:0]  an
);

                                                                                    //
==================================================================
=====
   //  Power-On Reset
                                                                                    //
==================================================================
=====
   reg [3:0] por_cnt = 4'hF;
   reg       por_rst = 1'b1;

   always @(posedge clk) begin
      if (por_cnt != 4'h0) begin
         por_cnt <= por_cnt - 4'h1;
         por_rst <= 1'b1;
      end else begin
         por_rst <= 1'b0;
      end
   end

   wire rst_int = rst_btn | por_rst;
```

```
                                                                    //
==================================================================
=====
  //  Button Debounce + Edge Detect
                                                                    //
==================================================================
=====
  wire step_debounced;
  wire step_pulse;

  debounce db_step (
     .clk    (clk),
     .rst    (rst_int),
     .btn_in (btn_step),
     .btn_out(step_debounced)
  );

  edge_detect ed_step (
     .clk      (clk),
     .rst      (rst_int),
     .signal_in(step_debounced),
     .pulse_out(step_pulse)
  );

                                                                    //
==================================================================
=====
  //  FSM State Encoding
                                                                    //
==================================================================
=====
  localparam [3:0]
     S_IDLE     = 4'd0,
     S_W_X1     = 4'd1,
     S_W_X2     = 4'd2,
     S_W_X3     = 4'd3,
     S_READ_ADD = 4'd4,
     S_WRES_ADD = 4'd5,
     S_READ_SUB = 4'd6,
     S_WRES_SUB = 4'd7,
     S_READ_AND = 4'd8,
     S_WRES_AND = 4'd9,
     S_READ_OR  = 4'd10,
     S_WRES_OR  = 4'd11,
```

```
    S_DONE     = 4'd12;

localparam [31:0]
    CONST_A = 32'h10101010,
    CONST_B = 32'h01010101;

reg [3:0] fsm_state;
reg       r_WE;
reg  [4:0] r_rs1, r_rs2, r_rd;
reg [31:0] r_WD;
reg  [3:0] r_ALUCtrl;

always @(posedge clk or posedge rst_int) begin
    if (rst_int) begin
        fsm_state <= S_IDLE;
        r_WE      <= 1'b0;
        r_rs1     <= 5'd0;
        r_rs2     <= 5'd0;
        r_rd      <= 5'd0;
        r_WD      <= 32'b0;
        r_ALUCtrl <= 4'b0000;
    end else if (sw_phys[3] && step_pulse) begin
        case (fsm_state)
            S_IDLE: begin
                fsm_state <= S_W_X1;
                r_WE      <= 1'b1;
                r_rd      <= 5'd1;
                r_WD      <= CONST_A;
                r_ALUCtrl <= 4'b0000;
            end
            S_W_X1: begin
                fsm_state <= S_W_X2;
                r_WE      <= 1'b1;
                r_rd      <= 5'd2;
                r_WD      <= CONST_B;
                r_ALUCtrl <= 4'b0000;
            end
            S_W_X2: begin
                fsm_state <= S_W_X3;
                r_WE      <= 1'b0;
                r_ALUCtrl <= 4'b0000;
            end
            S_W_X3: begin
                fsm_state <= S_READ_ADD;
                r_rs1     <= 5'd1;
```

```
        r_rs2     <= 5'd2;
        r_ALUCtrl <= 4'b0000;
    end
    S_READ_ADD: begin
        fsm_state <= S_WRES_ADD;
        r_WE      <= 1'b1;
        r_rd      <= 5'd4;
        r_WD      <= alu_result;
        r_ALUCtrl <= 4'b0000;
    end
    S_WRES_ADD: begin
        fsm_state <= S_READ_SUB;
        r_WE      <= 1'b0;
        r_ALUCtrl <= 4'b0001;
    end
    S_READ_SUB: begin
        fsm_state <= S_WRES_SUB;
        r_WE      <= 1'b1;
        r_rd      <= 5'd5;
        r_WD      <= alu_result;
        r_ALUCtrl <= 4'b0001;
    end
    S_WRES_SUB: begin
        fsm_state <= S_READ_AND;
        r_WE      <= 1'b0;
        r_ALUCtrl <= 4'b0010;
    end
    S_READ_AND: begin
        fsm_state <= S_WRES_AND;
        r_WE      <= 1'b1;
        r_rd      <= 5'd6;
        r_WD      <= alu_result;
        r_ALUCtrl <= 4'b0010;
    end
    S_WRES_AND: begin
        fsm_state <= S_READ_OR;
        r_WE      <= 1'b0;
        r_ALUCtrl <= 4'b0011;
    end
    S_READ_OR: begin
        fsm_state <= S_WRES_OR;
        r_WE      <= 1'b1;
        r_rd      <= 5'd7;
        r_WD      <= alu_result;
        r_ALUCtrl <= 4'b0011;
```

```
            end
            S_WRES_OR: begin
               fsm_state <= S_DONE;
               r_WE      <= 1'b0;
            end
            S_DONE: begin
               fsm_state <= S_IDLE;
            end
            default: fsm_state <= S_IDLE;
         endcase
      end
   end

                                                                    //
==================================================================
=====
   //  Core Components
                                                                    //
==================================================================
=====
   wire [31:0] rd1, rd2;
   wire [31:0] alu_result;
   wire        zero_flag;

   RegisterFile rf_inst (
      .clk(clk), .rst(rst_int), .WriteEnable(r_WE),
      .rs1(r_rs1), .rs2(r_rs2), .rd(r_rd), .WriteData(r_WD),
      .ReadData1(rd1), .ReadData2(rd2)
   );

   ALU alu_inst (
      .A(rd1), .B(rd2), .ALUControl(r_ALUCtrl),
      .ALUResult(alu_result), .Zero(zero_flag)
   );

                                                                    //
==================================================================
=====
   //  Seven-Segment Display
                                                                    //
==================================================================
=====
   wire [15:0] seg_value = {fsm_state, 3'b000, zero_flag, alu_result[7:0]};

   seven_seg_hex seg_inst (
```

```
        .clk(clk), .value(seg_value), .seg(seg), .an(an)
    );

                                                                                //
==================================================================
=====
    //  Peripheral Modules
                                                                                //
==================================================================
=====

    // Switches module reads physical switch inputs
    switches sw_inst (
        .clk(clk),
        .rst(rst_int),
        .btns(4'b0),
        .writeData(32'b0),
        .writeEnable(1'b0),
        .readEnable(1'b1),
        .memAddress(30'b0),
        .switches(sw_phys),
        .readData()
    );

    // Leds module writes ALU result to physical LEDs
    leds led_inst (
        .clk(clk),
        .rst(rst_int),
        .writeData(alu_result),
        .writeEnable(1'b1),
        .readEnable(1'b0),
        .memAddress(30'b0),
        .readData(),
        .leds(led_phys)
    );

endmodule
```

LEDS AND SWITCHES (FROM LAB5):

```
`timescale 1ns/1ps

module leds(
    input clk,
    input rst,
    input [31:0] writeData,
    input writeEnable,
```

```
    input readEnable,
    input [29:0] memAddress,
    output reg [31:0] readData = 0,
    output reg [15:0] leds
);

always @(posedge clk or posedge rst)
begin
    if (rst)
        leds <= 16'b0;
    else if (writeEnable)
        leds <= writeData[15:0];
end

endmodule
```

SWITCHES:

```
`timescale 1ns/1ps

module switches(
    input clk, rst,
    input [15:0] btns,
    input [31:0] writeData,
    input writeEnable,
    input readEnable,
    input [29:0] memAddress,
    input [15:0] switches,
    output reg [31:0] readData
);

always @(posedge clk or posedge rst)
begin
    if (rst)
        readData <= 32'b0;
    else if (readEnable)
        readData <= {16'b0, switches};
end

endmodule
```

Debouncer:

```
module debounce #(parameter DB_CNT = 20) (
    input  wire clk,
    input  wire rst,
    input  wire btn_in,
```

```
    output reg  btn_out
);
    reg [DB_CNT-1:0] shift;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            shift   <= 0;
            btn_out <= 0;
        end else begin
            shift   <= {shift[DB_CNT-2:0], btn_in};
            btn_out <= &shift;
        end
    end
Endmodule
```

Edge_detect:

```
module edge_detect (
    input  wire clk,
    input  wire rst,
    input  wire signal_in,
    output wire pulse_out
);
    reg prev;
    always @(posedge clk or posedge rst)
        if (rst) prev <= 1'b0;
        else     prev <= signal_in;
    assign pulse_out = signal_in & ~prev;
endmodule
```

module seven_seg_hex:

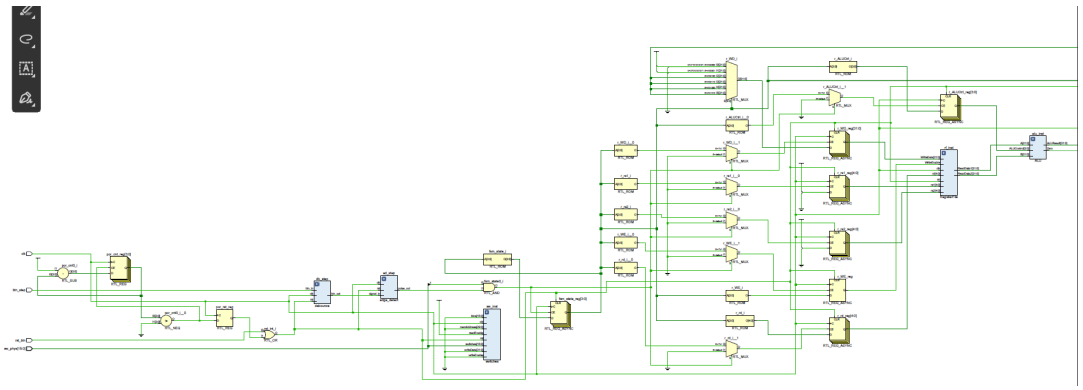
Functionality: This module drives a 4-digit multiplexed seven-segment display, converting a 16-bit input into four hexadecimal characters (0-F).
Time-Multiplexing: Instead of driving all 4 digits at once (which would require too many pins), it cycles through them one by one. Using bits 17 and 16 of a `refresh` counter, it switches digits at about **380Hz**. This is fast enough that the human eye sees all four digits as being "on" constantly without flickering.
Active-Low Logic: The segments are configured for active-low operation. Pulling an anode pin to `0` enables that specific digit, and pulling a segment pin to `0` turns that segment on.
Decoding: A 4-bit nibble is extracted from the 16-bit value based on the current scan state and passed through a combinational case statement to determine the correct pattern for the segments (a-g).

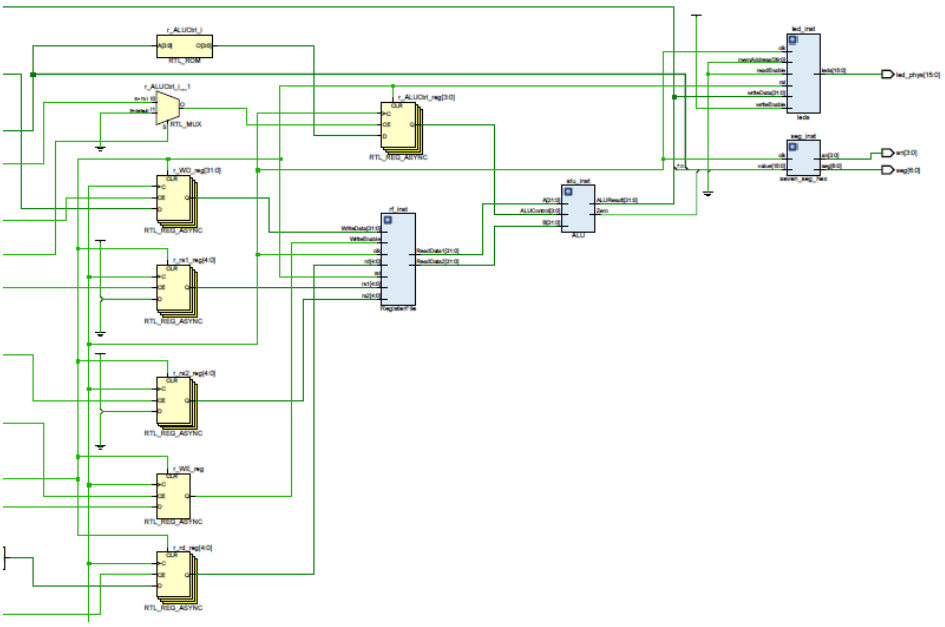
Refresh Rate: By using a 100MHz system clock and an 18-bit counter, the full display refresh occurs at roughly **95Hz**, ensuring a stable and clear output on the hardware.

```
`timescale 1ns / 1ps
module seven_seg_hex (
   input  wire       clk,       // System Clock (100MHz)
   input  wire [15:0] value,     // Data to show (4 hex digits)
   output reg  [6:0]  seg,       // Segment pins g-a (low = ON)
   output reg  [3:0]  an         // Anode select pins (low = ON)
);

   // ----------------------------------------------------------------
   // Refresh Counter
   // Using bits 17 and 16 to cycle through digits at ~380Hz.
   // This is fast enough to fool the eye so it doesn't flicker.
   // ----------------------------------------------------------------
   reg [17:0] refresh;
   always @(posedge clk)
      refresh <= refresh + 1;

   // ----------------------------------------------------------------
   // Digit Multiplexer
   // Pick one 4-bit nibble at a time based on the refresh counter.
   // Only one anode is pulled low at a time to enable that specific digit.
   // ----------------------------------------------------------------
   reg [3:0] nibble;

   always @(*) begin
      case (refresh[17:16])
         2'b00: begin nibble = value[ 3: 0]; an = 4'b1110; end  // Rightmost digit
         2'b01: begin nibble = value[ 7: 4]; an = 4'b1101; end  // 2nd digit
         2'b10: begin nibble = value[11: 8]; an = 4'b1011; end  // 3rd digit
         2'b11: begin nibble = value[15:12]; an = 4'b0111; end  // Leftmost digit
      endcase
   end

   // ----------------------------------------------------------------
   // Hex to 7-Segment Decoder
   // Maps the 4-bit nibble to the physical segment pattern.
   // Format: seg[6:0] = {g, f, e, d, c, b, a}
   // Logic is active-low: 0 turns the segment ON, 1 turns it OFF.
   // ----------------------------------------------------------------
```

```
    always @(*) begin
        case (nibble)
            4'h0: seg = 7'b1000000; // 0
            4'h1: seg = 7'b1111001; // 1
            4'h2: seg = 7'b0100100; // 2
            4'h3: seg = 7'b0110000; // 3
            4'h4: seg = 7'b0011001; // 4
            4'h5: seg = 7'b0010010; // 5
            4'h6: seg = 7'b0000010; // 6
            4'h7: seg = 7'b1111000; // 7
            4'h8: seg = 7'b0000000; // 8
            4'h9: seg = 7'b0010000; // 9
            4'hA: seg = 7'b0001000; // A
            4'hB: seg = 7'b0000011; // B
            4'hC: seg = 7'b1000110; // C
            4'hD: seg = 7'b0100001; // D
            4'hE: seg = 7'b0000110; // E
            4'hF: seg = 7'b0001110; // F
            default: seg = 7'b1111111; // Display nothing
        endcase
    end

endmodule
```

RTL SCHEMATIC:





Power-On Reset & Input Conditioning: On the far left, there is a 4-bit counter logic that handles the initial power-on reset (`por_cnt`). This is combined with the physical `rst_btn` to create the global `rst_int` signal. The `btn_step` input goes through the `db_step` (debounce) and `ed_step` (edge detect) blocks to ensure a single clean pulse for each button press.

FSM Logic (State Machine): The central part of the schematic features several ROM and MUX blocks that implement the FSM. This state machine controls the flow of data by generating the necessary addresses and control signals based on the current `fsm_state`. You can see the `fsm_state_reg` feeding back into the logic to determine the next step in the sequence.

Register File & ALU Datapath: The large blue block in the center-right is the `RegisterFile`. It takes in `r_rs1`, `r_rs2`, and `r_rd` addresses from the FSM. The outputs (`ReadData1` and `ReadData2`) feed directly into the `ALU` block (`alu_inst`). The `ALUResult` then loops back to the `WriteData` port of the register file so results can be saved for the next state.

Switch Interface (Input): The `switches` module (`switch_interface`) is connected to the physical `sw_phys` inputs. In the code, `sw_phys[3]` acts as an "enable" for the FSM—unless this switch is flipped high, the FSM won't advance even if you press the step button.

LED Interface (Output): The `leds` module (`led_interface`) takes the current `ALUResult` and drives it to the physical `led_phys` pins. This gives a

direct 16-bit binary representation of whatever math operation (ADD, SUB, AND, OR) the FSM just completed.

Display Logic: At the bottom right, there is a MUX (`display_val_i`) that selects data for the seven-segment display. It combines the current FSM state, the ALU's `Zero` flag, and the lower 8 bits of the `alu_result` so you can debug the system's progress visually on the board.

Results on fpga:

FSM State (Hex ID)	Operation	Calculation (Hex)	Displayed Result
4 (Read) / 5 (Write)	ADD	`10101010 + 01010101`	x011
6 (Read) / 7 (Write)	SUB	`10101010 - 01010101`	x00F
8 (Read) / 9 (Write)	AND	`10101010 & 01010101`	x100
A (Read) / B (Write)	OR	`10101010 | 01010101`	x011
C (Read) / D (Write)	XOR	`10101010 ^ 01010101`	x011
E (Read) / F (Write)	SLL	`10101010 << 5`	x100

CONSTRAINT FILE:

```
# ==================================================================
# Basys 3 XDC for top_rf_alu
# Port names: clk, rst_btn, sw_phys[3:0], led_phys[15:0], seg[6:0], an[3:0]
# ==================================================================
```

```
# Voltage
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

# ---------------------------------------------------------------
# Clock - 100 MHz
# ---------------------------------------------------------------
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
clk]

# ---------------------------------------------------------------
# Reset button - BTNR right button (T17), active high
# ---------------------------------------------------------------
set_property PACKAGE_PIN T17 [get_ports rst_btn]
set_property IOSTANDARD LVCMOS33 [get_ports rst_btn]

# ---------------------------------------------------------------
# Step button - BTNC center button (U18)
# Press to advance FSM one state when sw_phys[3]=1
# ---------------------------------------------------------------
set_property PACKAGE_PIN U18 [get_ports btn_step]
set_property IOSTANDARD LVCMOS33 [get_ports btn_step]

# ---------------------------------------------------------------
# Slide switches SW0-SW3 -> sw_phys[0]-sw_phys[3]
#   sw_phys[3] = demo FSM enable (SW3, leftmost of the four)
# ---------------------------------------------------------------
set_property PACKAGE_PIN V17 [get_ports {sw_phys[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_phys[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw_phys[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_phys[1]}]
set_property PACKAGE_PIN W16 [get_ports {sw_phys[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_phys[2]}]
set_property PACKAGE_PIN W17 [get_ports {sw_phys[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_phys[3]}]

# ---------------------------------------------------------------
# LEDs LD0-LD15 -> led_phys[0]-led_phys[15]
#   led_phys[3:0]  = FSM state
#   led_phys[4]    = ALU Zero flag
#   led_phys[15:5] = alu_result[10:0]
# ---------------------------------------------------------------
set_property PACKAGE_PIN U16 [get_ports {led_phys[0]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[0]}]
set_property PACKAGE_PIN E19 [get_ports {led_phys[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[1]}]
set_property PACKAGE_PIN U19 [get_ports {led_phys[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[2]}]
set_property PACKAGE_PIN V19 [get_ports {led_phys[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[3]}]
set_property PACKAGE_PIN W18 [get_ports {led_phys[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[4]}]
set_property PACKAGE_PIN U15 [get_ports {led_phys[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[5]}]
set_property PACKAGE_PIN U14 [get_ports {led_phys[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[6]}]
set_property PACKAGE_PIN V14 [get_ports {led_phys[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[7]}]
set_property PACKAGE_PIN V13 [get_ports {led_phys[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[8]}]
set_property PACKAGE_PIN V3  [get_ports {led_phys[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[9]}]
set_property PACKAGE_PIN W3  [get_ports {led_phys[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[10]}]
set_property PACKAGE_PIN U3  [get_ports {led_phys[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[11]}]
set_property PACKAGE_PIN P3  [get_ports {led_phys[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[12]}]
set_property PACKAGE_PIN N3  [get_ports {led_phys[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[13]}]
set_property PACKAGE_PIN P1  [get_ports {led_phys[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[14]}]
set_property PACKAGE_PIN L1  [get_ports {led_phys[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_phys[15]}]

# ----------------------------------------------------------------
# 7-Segment cathodes seg[6:0]  (active low)
#   seg[0]=a  seg[1]=b  seg[2]=c  seg[3]=d
#   seg[4]=e  seg[5]=f  seg[6]=g
# ----------------------------------------------------------------
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
```

```
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]

# ---------------------------------------------------------------
# 7-Segment anodes an[3:0]  (active low)
#   an[0]=rightmost digit   an[3]=leftmost digit
# ---------------------------------------------------------------
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```